Object Oriented Software Development

Generics

Agenda & Reading

- Topics:
  - Generics
    - Introduction
    - Generic Methods
      - Standard
      - Restriction on type arguments

Non-exam

- Reading
  - The Java Tutorial:
    - http://docs.oracle.com/javase/tutorial/java/generics/index.html

Generis

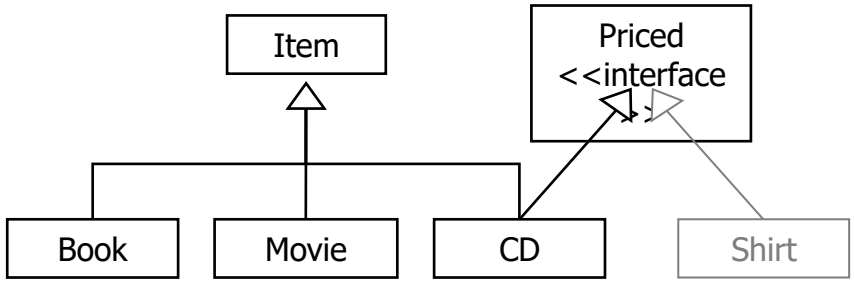
- What Are Generics
  - Generics abstract over Types
  - Classes, Interfaces and Methods can be **Parameterized by Types**
  - Generics provide increased **readability** and **type safety**
    - Some bugs are easier to detect than others.
      - Compile-time bugs, for example, can be detected early on;
      - Runtime bugs, however, can be much more problematic;
    - Generics add stability to your code by making more of your bugs detectable at **compile** time

```
public class Item {
  protected String title; ...
}
```

```
public class Movie extends Item {
  private int year; ...
}
```

```
public class Book extends Item {
  private String author; ...
}
```

```
public class CD extends Item implements Priced {
  private double price;
  ...
```




```
public interface Priced {
  public double getPrice();
}
```

1.Generis
Store a list of …

- Create a specialised List for each one?
  - Repetitive
  - Inefficient
  - Hardly scalable

```
public class MovieList {
  private List movies;
    ...
```

```
public class BookList  {
  private List books; ...
```

```
public class CDList {
  private List cds; ...
```

- With generics, we can make **List a generic type**
  - By parameterising List with the type Item we are guaranteed that any instance of this special List would only contain objects of **type T (or its subclasses)**
  - Using an array list of Items

```
List<Item> items = new ArrayList<Item>();
items.add(new Movie("Psycho",1960));
items.add(new Book("LOTR","Toklien"));
for (Item i: items)
  System.out.println(i.getTitle());
```

```
List<Item> items = new ArrayList<Item>();
items.add(new Movie("Psycho",1960));

items.add("Terminator");
```

Compile time error

Comparable – without Generics

```
public interface Comparable {
  public int compareTo(Object obj);
}
```

OLD Version!

- int compareTo() returns an integer result:
  - Returns negative if the object it is applied to is less than the argument
  - Returns zero if the object it is applied to is equal to the argument
  - Returns positive if the object it is applied to is greater than the argument
- Example: Person – compare by IRD number

```
public class Person implements Comparable {
  protected String irdNumber;
  ...
  public int compareTo(Object obj) {
    if (!(obj instanceof Person)) {
      throw new ClassCastException("Not a Person");
    }
    Person p = (Person) obj;
    return irdNumber.compareTo(p.getIRD());
  }
}
```

The parameter type is "Object", but need to do casting in order to compare two Person objects!

No compile time ERROR
But with Run-time error!

```
Person p1 = new Person("Stanley", "Smith", "123-456-789");
System.out.println(p1.compareTo(new Integer(1)));
```

Comparable With Generics

```
public interface Comparable<T> {
  public int compareTo(T o);
}
```

- Example: Person – compare by IRD number
  - Type parameter: Person
  - Type of the parameter variable in the compareTo method: Person
  - No casting is needed.
  - Direct access to the methods defined in the Person class
  - Provide Compile-time checking

```
public class Person implements Comparable<Person>
protected String irdNumber;
  ...
 public int compareTo(Person p) {
    return irdNumber.compareTo(p.getIRD());
  }
}
```

Type: Person

compile time ERROR!

```
Person p1 = new Person("Stanley", "Smith", "123-456-789");
System.out.println(p1.compareTo(new Integer(1)));
```

Comparable & Arrays.sort()

- Sorting an array of Objects
  - The Arrays.sort(Object[]) method requires the object type to implement the Comparable interface so that the array can be sorted according to natural ordering of its elements.
  - The natural ordering is defined by implementation of the compareTo() method which determines how the current object is compared to another (obj) of the same type.

```
Person[] p = new Person[3];
p[0] = new Person("Stanley", "Smith", "123-456-789");
p[1] = new Person("David" , "Clark", "999-222-111");
p[2] = new Person("Jane", "Graff", "245-123-345");

Arrays.sort(p);
System.out.println();
```

Exercise 1 Comparable

```
class Book {
  private String title, author;
  private int numOfPages=1;
  public Book(String title, String author, int pages) {
    this.title = title;
    this.author = author;
    numOfPages = pages;
  }
...
```

- Consider the following:

```
public interface Comparable<T> {
  public int compareTo(T o);
}
```

- modify the Book class. The Book class
  - implements the Comparable<Book> interface
  - implements the int compareTo(Book other) method
    - compare by the number of pages first, then compare by title

```
class Book implements Comparable<Book>{
  ...
  public int compareTo(Book b) {
     return numOfPages == b.numOfPages? ...
```

```
[Ready Player One(Tade Thompson, 150 pages), The Martian(Andy Weir,
150 pages), Fluency(Jennifer Foehner Wells, 321 pages)]
```

Example: TestGenericsApp

2.Generic Methods

- Define generic methods just as you define generic classes
  - Method declarations can be generic - that is, parameterized by one or more type parameters.

generic method

A type parameter:T

```
public static <T> void fromArrayToList(T[] a, List<T> c) {
  for (T o : a) {
    c.add(o); // correct
  }
}
```

- Type parameters can also be declared within method and constructor signatures to create generic methods and generic constructors.
  - But the type parameter's scope is limited to the method or constructor in which it's declared.

2.Generic Methods
Why to use Generic Method

- You can write a single generic method declaration that can be called with arguments of different types.
  - Based on the types of the arguments passed to the generic method, the compiler handles each method call appropriately. Following are the rules to define Generic Methods:
  - Note:
    - All generic method declarations have a type parameter section delimited by angle brackets (< and >) that precedes the method's return type (< T > in the next example).
    - Each type parameter section contains one or more type parameters separated by commas.

```
public static <T> void fromArrayToList(T[] a, List<T> c) {
   ...
}
```

Note: the parameter is an array of something and the second parameter is an array list of something

2.Generic Methods Examples

```
public static <T> void fromArrayToList(T[] a, List<T> c) {
  for (T o : a) {
    c.add(o);
  ...
```

- Define a static method named fromArrayToList(T
  - Take all elements in the first parameter array and insert them into the second parameter array list.
  - Note: You ***don't*** need to mention types when you call a method:

```
String[] sa = new String[]{"Hello", "World"};
List<String> cs = new ArrayList<String>();
cs.add("Morning");
fromArrayToList(sa, cs); // T inferred to be String
```

```
[Morning, Hello, World]
```

```
Item[] itemArray = new Item[2];
itemArray[0] = new Item("Hello");
itemArray[1] = new Book("LOTR","Tolkien");
List<Item> itemList2 = new ArrayList<Item>();
fromArrayToList(itemArray,itemList2);
```

```
[Item(Hello), Book: 'LOTR' by Tolkien]
```

The complete syntax for invoking the method:

```
TestGenericsApp.<String>fromArrayToList(mystrArray,myStrlist);
```


11

Exercise 2

- Define a static generic method named reverse(List<T> list)
  - returns a new list which elements are in reversed order from the original list.
    - Note: You can pass a list of String/points/movies … as a parameter to the method

```
public static <T> List<T> reverse(List<T> list) {


  return result;
}
```

```
List<String> words = new ArrayList<String>();
words.add("a");
words.add("b");
words.add("c");
words.add("d");
words.add("e");
List<String> sdrow = reverse(words);
System.out.println(sdrow);
```

```
List<Movie> movies = new ArrayList<Movie>();
...
List<Movie> seivom = reverse(movies);
System.out.println(seivom);
```

```
[e, d, c, b, a]
```

2.Generic Methods
Restriction on type arguments

- Bounded type parameters (e.g. <T extends Item>) allow restriction on type arguments
  - E.g. define a static method named startWithJ() that operates on Items only
    - accepts instances of Item or its subclasses only
    - returns a new list. The list contains all items that start with an "J"

```
List<Movie> moviesJ = startsWithJ(movies);
```

```
//List<String> wordsJ = startsWithJ(words);
```

COMPILE-TIME ERROR (String is not a subclass of Item)

  - To declare a bounded type parameter, list the type parameter's name, followed by the extends keyword, followed by its upper bound.

```
public static <T extends Item> List<T> startsWithJ(List<T> list) {
  ...
```

2.Generic Methods
Restriction on type arguments

- Solution:

```
public static <T extends Item> List<T> startsWithJ(List<T> list) {
  List<T> filtered = new ArrayList<T>();
  for (T item : list)
    if (item.getTitle().startsWith("J"))
      filtered.add(item);
  return filtered;
}
```

Notice how we can call `getTitle()` on `item`, because we know from the type parameter `T`'s declaration that it extends `Item`

```
List<Movie> movieList = new ArrayList<Movie>();
movieList.add(new Movie("Psycho",1960));
...
List<Movie> moviesJ = startsWithJ(movieList);
System.out.println(moviesJ);
```

```
[Movie: Jaws (1975), Movie: Jurassic Park (1993)]
```

Exercise 3
Generic Methods

```
class Item {
 ...
  public String getTitle()
 ...
class Book extends Item {
  ...
class Movie extends Item {
  ...
```

- Consider the above class definitions:
  - Define a static generic method
    - named getTitleToUpperCase(List<T> list)
    - accepts instances of Item or its subclasses only
    - returns a new string list. The list contains all titles in uppercase

```
public static <T extends        > List<String> getTitleToUpperCase(List<T> list)
{

}
```

```
List<Movie> movies = new ArrayList<Movie>();
...
List<String> titles =
getTitleToUpperCase(movies);
System.out.println(titles);
```

```
([PSYCHO, GODZILLA VS. KONG]
```